



جامعة الإمام عبد الرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY

Sprint 1

Logistics Paperwork Process Automation System (Web-Based Document Upload, Data Extraction, and Form Generation)

Course

BIS405 – Graduation Project

Area

Digital Systems and Solution Design

Team Name:

Team Members & IDs:

Norah Alshehri	2230004682
Bayader Alghamdi	2230003715
Shaykhah Mohsen	2230005764
Maha Alnafea	2230003005

Supervisor: Dr. Linah

Date: 3/13/2026

1. Increment Delivered

In this sprint, the team focused on establishing a scalable and secure backend architecture, ensuring that the foundation can support high-load logistics data processing and AI integration in future phases.

A. Database Infrastructure & Relational Modeling

The team designed and implemented a robust relational database using SQLAlchemy ORM. This abstraction layer allows the system to interact with the database using Python objects, ensuring code maintainability and database independence.

Database Structure

Browse Data

Edit Pragma

Execute SQL

Create Table

Create Index

Modify Table

Delete Table

Print

Refresh

Name	Type	Schema
Tables (3)		
cases		CREATE TABLE cases (case_id INTEGER NOT NULL, customer VARCHAR NOT NULL, service_type VARCHAR, status VARCHAR, created_at DATETIME, created_by INTEGER, PRIMARY KEY (case_id))
documents		CREATE TABLE documents (doc_id INTEGER NOT NULL, case_id INTEGER, doc_type VARCHAR, file_path VARCHAR NOT NULL, upload_time DATETIME, uploaded_by INTEGER, PRIMARY KEY (doc_id), FOREIGN KEY(case_id) REFERENCES cases (case_id))
users		CREATE TABLE users (id INTEGER NOT NULL, username VARCHAR, hashed_password VARCHAR NOT NULL, PRIMARY KEY (id))
Indices (4)		
ix_cases_case_id		CREATE INDEX ix_cases_case_id ON cases (case_id)
ix_documents_doc_id		CREATE INDEX ix_documents_doc_id ON documents (doc_id)
ix_users_id		CREATE INDEX ix_users_id ON users (id)
ix_users_username		CREATE UNIQUE INDEX ix_users_username ON users (username)
Views (0)		
Triggers (0)		

GET /cases/ Read Cases

POST /cases/ Create Case

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{  "customer": "Global Traders LLC",  "service_type": "Import",  "created_by": 1}
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{  "case_id": 0,  "customer": "string",  "service_type": "string",  "status": "string",  "created_at": "2026-03-01T02:10:58.324Z",  "created_by": 0}
```

Design Rationale & Data Integrity:

We adopted a normalized relational structure (Third Normal Form) to eliminate data redundancy and prevent update anomalies. By utilizing Foreign Key constraints, we ensured strict referential integrity between shipment cases and their associated documentation.

Table Architecture:

- **Users Table**

Stores unique user identifiers, validated email addresses, and metadata such as account creation dates and active status.

- **Cases Table**

Acts as the primary entity for shipment tracking. Each record includes a `case_id` (Primary Key), a descriptive title, and a relationship link to the owner (User) who initiated the shipment.

- **Documents Table**

Implements a one-to-many relationship with the Cases table. This allows each shipment case to host multiple related files (Invoices, Packing Lists, Customs Clearances) without duplicating shipment data.

Forward-Looking Attributes:

We integrated specific tracking fields like `upload_timestamp` for audit trails and a boolean `is_processed` flag. The latter is critical for the upcoming AI sprint, as it will trigger the data extraction engine for newly uploaded files.

B. Secure Authentication & Identity Management

To safeguard sensitive commercial logistics data, we developed a comprehensive security module using JWT (JSON Web Tokens) and Bcrypt.

POST

/register/

Register User

^

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

▼

Edit Value

Schema

```
{
  "username": "admin",
  "password": "admin"
}
```

username

*

required

string

admin

password

*

required

string(\$password)

.....

Code

Details

200

Response body

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIjZG1pb1IsInV4cCI6IjMjMzNDM4MDh0LGIIDEQU28eUg45nbkg74Q12MQRzNQVMuGxV810Fukcc",
  "token_type": "bearer"
}
```

Download

Response headers

```
content-length: 165
content-type: application/json
date: Sun, 01 Mar 2026 02:36:28 GMT
server: uvicorn
```

Bcrypt Password Hashing

To protect against data breaches and "Rainbow Table" attacks, we implemented Bcrypt hashing with a secure salt factor. Passwords are never stored or transmitted in plain text.

JWT Session Lifecycle:

- **Stateless Authentication:** We utilized JWT to maintain a stateless server environment, where the user receives a signed Bearer Token upon successful login.
- **Strict Expiration Logic:** To mitigate "Replay Attacks," tokens are configured with a 30-minute expiration window, forcing a re-authentication process to maintain a secure session.

Comprehensive Testing & Edge Case Validation:

- **Negative Testing:** We verified that attempts to access protected routes with malformed or expired tokens result in a 401 Unauthorized response.
- **Duplicate & Input Validation:** The registration logic performs real-time checks to prevent account duplication and ensures that only valid corporate email formats are accepted.

C. RESTful API Development & Advanced File Handling

We leveraged the asynchronous capabilities of **FastAPI** to build high-performance endpoints that facilitate seamless communication between the client and the server.

Full CRUD Lifecycle:

The system supports the complete lifecycle of shipment management. This includes dynamic status updates (e.g., changing a case status from 'Pending' to 'In Review') and soft-deletion mechanisms to preserve historical data for auditing.

POST	/register/	Register User	▼
POST	/login/	Login	☑
GET	/cases/	Read Cases	▼
POST	/cases/	Create Case	▼
PUT	/cases/{case_id}	Update Case Status	▼
DELETE	/cases/{case_id}	Delete Case	▼
POST	/cases/{case_id}/documents/	Upload Document	▼

Enterprise-Grade File Upload System:

- **Multipart/Form-Data Validation:** The `POST /upload` endpoint is restricted to handle only authorized MIME types (PDF, PNG, JPG). This prevents the execution of malicious scripts on the server.
- **Infrastructure Constraints:** A strict 5MB size limit is enforced per file to prevent Denial-of-Service (DoS) attacks and ensure storage efficiency.
- **Secure Storage Architecture:** Files are stored in a dedicated, isolated directory named `/uploaded_docs/`. The database only maintains sanitized, absolute file paths, ensuring that the physical files are decoupled from the database records for better backup management.

2. Progress Evidence & Sprint Analysis

The team adopted an Agile-Scrum methodology, utilizing GitHub Projects as the primary engine for task tracking and sprint management. This ensured that every technical increment was documented, assigned, and verified through a transparent workflow.

A. Strategic Task Distribution & Specialization

To maximize productivity, tasks were divided into two parallel workstreams based on the technical strengths of the team members:

- Security & System Routing (Norah & Bayader): This sub-team focused on the critical "Entry Points" of the application. They were responsible for building the `auth.py` module and the core routing logic in `main.py`, ensuring that all subsequent features would inherit a secure-by-design architecture.
- Data Modeling & Operational Services (Shaykhah & Maha): This sub-team managed the "Data Backbone". Their efforts were concentrated on translating logistics requirements into the `models.py` schema and implementing the Shipment CRUD services, ensuring that data persistence was both reliable and scalable.

B. Quantitative Effort Analysis

The sprint effort was distributed strategically to prioritize the system's integrity:

- Authentication & Security (40%): This represented the largest single effort block. The rationale was that establishing a foolproof security gate early on would prevent costly architectural changes in future sprints.
- Database Infrastructure (25%): Effort was spent on perfecting the relational links between cases and documents to support automated workflows.
- Core CRUD & API Development (35%): Once the security and database layers were stable, the team focused on building the functional endpoints that allow users to interact with shipment data.

C. Technical Reflections & Challenges

A key part of our sprint was overcoming technical "blockers" through collaborative research:

- The Challenge: A significant hurdle occurred during the integration of FastAPI with SQLAlchemy, specifically regarding Asynchronous Database Connections. We initially encountered "Connection Leak" errors and database locks when trying to perform simultaneous document uploads while querying shipment cases.
- The Resolution: The team conducted a technical review and resolved the issue by implementing a Centralized Session Management Utility in `database.py`. We introduced a Dependency Injection pattern (the `get_db` function), which ensures that database connections are scoped correctly to each request and terminated properly upon completion.
- Key Learning: This challenge taught us the importance of asynchronous programming in modern APIs. This solution has greatly improved code reusability and system performance, providing a stable foundation for the high-concurrency data extraction tasks scheduled for Sprint 2.

3. Next Sprint Plan

Sprint Goal:

Transition the system from a static infrastructure to an intelligent automation platform by initiating core data extraction and developing the interactive user interface.

Task 1: AI-Driven Document Data Extraction Logic

Objective:

To research, develop, and test specialized extraction rules designed to parse unstructured text from standard logistics documents into structured data formats.

Technical Focus:

We will focus on implementing pattern-matching and Natural Language Processing (NLP) techniques to identify key logistics entities such as Vendor Names, Invoice Numbers, Total Amounts, and Shipment Dates.

Measurable Deliverables:

1. Functioning Extraction Algorithm Prototype: A Python-based script integrated with the existing FastAPI backend that can process at least two distinct document templates (e.g., Commercial Invoices and Bills of Lading).
2. Standardized JSON Output: A sample JSON schema that demonstrates successful field mapping. For example, the algorithm should output: `{"invoice_no": "INV-99", "total_weight": "500kg", "currency": "USD"}`.

Assigned to: Norah Alshehri & Bayader Alghamdi.

Task 2: Front-end Review & Data Validation Interface

Objective:

To design and implement a responsive user interface that serves as a "Human-in-the-loop" verification layer for the automated extraction results.

Technical Focus:

The UI will focus on a "Side-by-Side" view, where the user can see the uploaded PDF/Image on one side and the extracted data fields on the other for quick auditing.

Measurable Deliverables:

1. High-Fidelity Wireframes: Detailed UI/UX designs for the "Review & Edit" screen, showcasing the workflow from document selection to final data confirmation.
2. Functional UI Component: A front-end component (React/Vue) that fetches extraction results from the backend and allows users to manually correct any inaccuracies before saving the record to the Documents table.

Assigned to: Shaykhah Mohsen & Maha Alnafea.

Expected Sprint Outcome

By the end of Sprint 2, the system will not only store documents but will also "understand" their content, providing a significant step toward full logistics automation. We expect to demonstrate a complete "Upload-to-Extraction-to-Review" cycle for the supervisor's evaluation.